

WEEK 4–BOOKING AND BOOKING DETAILS MANAGEMENT SYSTEM–MOVIE TICKET

Scenario:

The Movie Ticket Booking Management System is designed to manage customer bookings for different movies. The system stores information about customers, their bookings, and the individual movies included in each booking.

The BOOKINGS table stores the main information about a booking, such as the customer who made the booking, booking date, total amount and booking status. The BOOKING_DETAILS table stores information about the movies included in each booking, such as movie ID, number of tickets and ticket price.

This system helps maintain booking information accurately and establishes relationships between customers, bookings and movies.

1. Design BOOKING and BOOKING_DETAILS Tables

BOOKINGS Table: The BOOKINGS table stores the main information related to customer movie bookings.

Attributes:

- Booking_ID – Unique identifier for each booking.
- Customer_ID – Identifies the customer who made the booking.
- Booking_Date – Stores the date on which the booking was made.
- Total_Amount – Stores the total cost of the booking.
- Booking_Status – Stores the current status of the booking, such as Confirmed, Pending or Completed.

BOOKING_DETAILS Table: The BOOKING_DETAILS table stores information about the individual movies included in a booking.

Attributes:

- Booking_Detail_ID – Unique identifier for each booking detail.
- Booking_ID – Identifies the corresponding booking.
- Movie_ID – Identifies the movie selected.
- Quantity – Stores the number of tickets booked.
- Price – Stores the price of the selected movie ticket.

```
SQL> CREATE TABLE BOOKINGS (Booking_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Booking_Date DATE, Total_Amount NUMBER(10,2), Booking_Status VARCHAR2(20));
Table created.
```

```
SQL> CREATE TABLE BOOKING_DETAILS (Booking_Detail_ID VARCHAR2(10), Booking_ID VARCHAR2(10), Movie_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));
Table created.
```

2. Establish Relationships:

Customer and Booking

One customer can make multiple bookings, while each booking belongs to one customer.

Relationship: Customer → Bookings = One-to-Many

The Customer_ID in the BOOKINGS table establishes this relationship.

Booking and Booking Details

One booking can contain multiple booking details, while each booking detail belongs to one booking.

Relationship:Bookings → Booking_Details = One-to-Many

The Booking_ID in BOOKING_DETAILS connects each booking detail with its corresponding booking.

Product and Booking Details

The BOOKING_DETAILS table is connected to the PRODUCT table through Movie_ID.

Relationship:Product → Booking_Details

This identifies the movie selected by the customer.

3.Apply Constraints

Constraints are used to maintain the accuracy and integrity of the database.

Primary Key

Primary keys uniquely identify records.

- Booking_ID → Primary Key of BOOKINGS
- Booking_Detail_ID → Primary Key of BOOKING_DETAILS

```
SQL> ALTER TABLE BOOKINGS ADD CONSTRAINT PK_Bookings PRIMARY KEY (Booking_ID);  
Table altered.
```

```
SQL> ALTER TABLE BOOKING_DETAILS ADD CONSTRAINT PK_BookingDetails PRIMARY KEY (Booking_Detail_ID);  
Table altered.
```

Foreign Key

Foreign keys establish relationships between tables.

- Customer_ID in BOOKINGS references CUSTOMER.
- Booking_ID in BOOKING_DETAILS references BOOKINGS.
- Movie_ID in BOOKING_DETAILS references PRODUCT.

```
SQL> ALTER TABLE BOOKINGS ADD CONSTRAINT FK_Bookings_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);  
Table altered.
```

```
SQL> ALTER TABLE BOOKING_DETAILS ADD CONSTRAINT FK_BD_Movie FOREIGN KEY (Movie_ID) REFERENCES PRODUCT(Product_ID);  
Table altered.
```

NOT NULL Constraint:

NOT NULL ensures that important information cannot be left empty.

Important fields include:

- Customer ID
- Booking Date
- Booking Status
- Quantity

- Price

```
SQL> ALTER TABLE BOOKINGS MODIFY Booking_Date NOT NULL;

Table altered.
```

CHECK Constraint

CHECK constraints ensure that only valid values are entered.

- Quantity must be greater than 0.
- Price must be greater than 0.
- Total Amount must be greater than or equal to 0.

```
SQL> ALTER TABLE BOOKINGS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);

Table altered.
```

```
SQL> ALTER TABLE BOOKINGS ADD CONSTRAINT CHK_Status CHECK (Booking_Status IN ('Pending','Confirmed','Completed'));

Table altered.
```

```
SQL> ALTER TABLE BOOKING_DETAILS ADD CONSTRAINT CHK_Quantity CHECK (Quantity > 0);

Table altered.
```

```
SQL> ALTER TABLE BOOKING_DETAILS ADD CONSTRAINT CHK_Booking_Price CHECK (Price > 0);

Table altered.
```

DESCRIBE THE STRUCTURE OF THE TABLES:

```
SQL> desc bookings;
Name                               Null?    Type
-----
BOOKING_ID                         NOT NULL VARCHAR2(10)
CUSTOMER_ID                        VARCHAR2(10)
BOOKING_DATE                       NOT NULL DATE
TOTAL_AMOUNT                       NUMBER(10,2)
BOOKING_STATUS                     VARCHAR2(20)
```

```
SQL> desc booking_details;
Name                               Null?    Type
-----
BOOKING_DETAIL_ID                 NOT NULL VARCHAR2(10)
BOOKING_ID                       VARCHAR2(10)
MOVIE_ID                         VARCHAR2(10)
QUANTITY                         NUMBER
PRICE                           NUMBER(10,2)
```

4.Insert Values

Sample records are inserted into the BOOKINGS and BOOKING_DETAILS tables.

The BOOKINGS table contains:

- Booking ID
- Customer ID
- Booking Date

- Total Amount
- Booking Status

```
SQL> INSERT INTO BOOKINGS VALUES('B001','C001',TO_DATE('15-09-2026','DD-MM-YYYY'),360.00,'Confirmed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B002','C002',TO_DATE('17-09-2026','DD-MM-YYYY'),300.00,'Pending');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B003','C003',TO_DATE('19-09-2026','DD-MM-YYYY'),480.00,'Completed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B004','C004',TO_DATE('21-09-2026','DD-MM-YYYY'),400.00,'Confirmed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B005','C005',TO_DATE('23-09-2026','DD-MM-YYYY'),340.00,'Pending');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B006','C006',TO_DATE('24-09-2026','DD-MM-YYYY'),440.00,'Completed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B007','C007',TO_DATE('25-09-2026','DD-MM-YYYY'),320.00,'Confirmed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B008','C008',TO_DATE('26-09-2026','DD-MM-YYYY'),360.00,'Pending');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B009','C009',TO_DATE('27-09-2026','DD-MM-YYYY'),400.00,'Completed');
1 row created.

SQL> INSERT INTO BOOKINGS VALUES('B010','C010',TO_DATE('28-09-2026','DD-MM-YYYY'),340.00,'Confirmed');
1 row created.
```

SELECT BOOKINGS TABLE:

```
SQL> SELECT * FROM BOOKINGS;

BOOKING_ID  CUSTOMER_I  BOOKING_D  TOTAL_AMOUNT  BOOKING_STATUS
-----
B001        C001        15-SEP-26        360 Confirmed
B002        C002        17-SEP-26        300 Pending
B003        C003        19-SEP-26        480 Completed
B004        C004        21-SEP-26        400 Confirmed
B005        C005        23-SEP-26        340 Pending
B006        C006        24-SEP-26        440 Completed
B007        C007        25-SEP-26        320 Confirmed
B008        C008        26-SEP-26        360 Pending
B009        C009        27-SEP-26        400 Completed
B010        C010        28-SEP-26        340 Confirmed

10 rows selected.
```

The BOOKING_DETAILS table contains:

- Booking Detail ID
- Booking ID
- Movie ID

- Quantity
- Price

```
SQL> INSERT INTO BOOKING_DETAILS VALUES('BD001','B001','P001',50,180);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD002','B002','P002',40,150);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD003','B003','P003',35,160);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD004','B004','P004',45,150);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD005','B005','P005',30,180);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD006','B006','P006',50,200);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD007','B007','P007',25,220);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD008','B008','P008',40,160);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD009','B009','P009',50,200);
1 row created.

SQL> INSERT INTO BOOKING_DETAILS VALUES('BD010','B010','P010',35,170);
1 row created.
```

SELECT BOOKING_DETAILS TABLE:

```
SQL> SELECT * FROM BOOKING_DETAILS;
```

BOOKING_DE	BOOKING_ID	MOVIE_ID	QUANTITY	PRICE
BD001	B001	P001	50	180
BD002	B002	P002	40	150
BD003	B003	P003	35	160
BD004	B004	P004	45	150
BD005	B005	P005	30	180
BD006	B006	P006	50	200
BD007	B007	P007	25	220
BD008	B008	P008	40	160
BD009	B009	P009	50	200
BD010	B010	P010	35	170

```
10 rows selected.
```

Reports

Customer Booking History

The Customer Booking History report displays the booking information associated with each customer.

It displays:

- Customer Name
- Booking ID
- Booking Date
- Total Amount
- Booking Status

```
SQL> SELECT C.CUSTOMER_NAME,B.BOOKING_ID,B.BOOKING_DATE,B.TOTAL_AMOUNT FROM CUSTOMER C JOIN BOOKINGS B ON C.CUSTOMER_ID=B.CUSTOMER_ID;
```

CUSTOMER_NAME	BOOKING_ID	BOOKING_D	TOTAL_AMOUNT
Arun	B001	15-SEP-26	360
Priya	B002	17-SEP-26	300
Kavi	B003	19-SEP-26	480
Rahul	B004	21-SEP-26	400
Divya	B005	23-SEP-26	340
Akshaya	B006	24-SEP-26	440
Vijay	B007	25-SEP-26	320
Reena	B008	26-SEP-26	360
Prithvi	B009	27-SEP-26	400
Riya	B010	28-SEP-26	340

10 rows selected.

Movie-wise Booking Report

The Movie-wise Booking Report provides information about the booking frequency and number of tickets for each movie.

It finds:

- Number of times each movie was booked.
- Total number of tickets booked for each movie.

```
SQL> SELECT P.PRODUCT_NAME AS MOVIE_NAME,COUNT(BD.BOOKING_DETAIL_ID) AS NUMBER_OF_BOOKINGS,SUM(BD.QUANTITY) AS TOTAL_TICKETS FROM PRODUCT P JOIN BOOKING_DET  
AILS BD ON P.PRODUCT_ID=BD.MOVIE_ID GROUP BY P.PRODUCT_NAME;
```

MOVIE_NAME	NUMBER_OF_BOOKINGS	TOTAL_TICKETS
The Lion King	1	35
Avengers	1	50
Toy Story	1	40
Good Night	1	40
Love Today	1	45
Interstellar	1	25
Leo	1	50
Conjuring	1	30
96	1	35
Vikram	1	50

10 rows selected.

Customer Booking Analysis

The Customer Booking Analysis report analyzes the booking activity of customers.

It finds:

- Total number of bookings by each customer.
- Total spending by each customer.
- Average booking value.

```
SQL> SELECT C.CUSTOMER_NAME,COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS,SUM(B.TOTAL_AMOUNT) AS TOTAL_SPENDING,AVG(B.TOTAL_AMOUNT) AS AVERAGE_BOOKING FROM CUSTOMER
C JOIN BOOKINGS B ON C.CUSTOMER_ID=B.CUSTOMER_ID GROUP BY C.CUSTOMER_NAME;
```

CUSTOMER_NAME	TOTAL_BOOKINGS	TOTAL_SPENDING	AVERAGE_BOOKING
Prithvi	1	400	400
Reena	1	360	360
Akshaya	1	440	440
Priya	1	300	300
Rahul	1	400	400
Divya	1	340	340
Riya	1	340	340
Kavi	1	480	480
Vijay	1	320	320
Arun	1	360	360

10 rows selected.

CRUD Operations

CRUD stands for Create, Read, Update and Delete. These are the four basic operations used to manage data in a database. In this system, CRUD operations are performed on the BOOKING_DETAILS table.

Create

```
SQL> INSERT INTO BOOKING_DETAILS VALUES ('BD011','B001','P003',2,160);
```

1 row created.

Read

```
SQL> SELECT * FROM Booking_Details;
```

BOOKING_DE	BOOKING_ID	MOVIE_ID	QUANTITY	PRICE
BD001	B001	P001	50	180
BD002	B002	P002	40	150
BD003	B003	P003	35	160
BD004	B004	P004	45	150
BD005	B005	P005	30	180
BD006	B006	P006	50	200
BD007	B007	P007	25	220
BD008	B008	P008	40	160
BD009	B009	P009	50	200
BD010	B010	P010	35	170
BD011	B001	P003	3	160

11 rows selected.

Update

```
SQL> UPDATE BOOKING_DETAILS SET QUANTITY=3 WHERE BOOKING_DETAIL_ID='BD011';
```

1 row updated.

Delete

```
SQL> DELETE FROM BOOKING_DETAILS WHERE BOOKING_DETAIL_ID='BD011';
```

1 row deleted.

Conclusion:

The Movie Ticket Booking Management System was successfully implemented using the concepts covered in Week 4. The BOOKINGS and BOOKING_DETAILS tables were created with appropriate primary keys, foreign keys, NOT NULL constraints and CHECK constraints.

Relationships were established between customers, bookings, booking details and movies. Sample booking data was inserted, and SQL joins were used to generate customer booking history, movie-wise booking reports and customer booking analysis. CRUD operations were also performed on booking details to manage the data effectively. This follows the structure and objectives of the Week 4 material.